

Configuration Performance Tuning with a Random-Forest Sequential Model

Intelligent Software Engineering Coursework Report

1. Introduction

This project builds a limited-budget intelligent software engineering tool for configuration performance tuning in the Lab 3 setting. The core challenge is to find strong configurations while treating every measurement as expensive, rather than exhaustively traversing the full configuration space.

The baseline is Random Search and the proposed method is Random-Forest Sequential Model-Based Optimisation (RF-SMBO). At budget 50, RF-SMBO lowers the overall mean normalised gap from 0.0487 to 0.0113, a reduction of 76.87%. However, this result is not uniform across all systems: the overall improvement is driven mainly by strong gains on LLVM and x264, while RF-SMBO underperforms Random Search on 7z.

2. Related Work

Random Search is a suitable baseline for black-box configuration optimisation because it is simple, fair under a fixed budget, and often stronger than naive exhaustive alternatives in high-dimensional spaces [1]. In the present coursework it provides a transparent reference point for understanding whether a learned search policy adds value.

Sequential Model-Based Optimisation (SMBO) improves on unguided search by fitting a surrogate on previous measurements and using it to decide which configuration to evaluate next [2].

Bayesian optimisation follows the same high-level idea and emphasises the balance between exploitation and uncertainty-driven exploration [3]. This budget-aware perspective closely matches the Lab 3 objective, where only a small number of measurements may be spent.

Random-forest surrogates are attractive in configurable software systems because they can model non-linear option interactions without heavy feature engineering, while remaining robust on mixed discrete or integer-like search spaces. Prior studies in software-system tuning and search-based software engineering show that uncertainty-aware or surrogate-guided methods can outperform purely random search in expensive tuning tasks [4], [5].

3. Solution

The valid rows in each dataset define the candidate configuration space. A runtime value is not used to rank all candidates in advance; instead, it is revealed only when a row is selected for measurement. RF-SMBO begins with six random measurements, retrains a surrogate after every new observation, and then uses that surrogate to choose the next valid configuration.

The surrogate is scikit-learn's RandomForestRegressor, which is a meta-estimator that averages the predictions of multiple decision-tree regressors. This is a natural fit here because averaging improves stability, helps control over-fitting, and still allows the model to capture non-linear interactions between configuration options.

Algorithm 1: RF-SMBO for Configuration Performance Tuning

1. Initialise the observed set with six random configurations.
2. Measure those configurations and store the revealed runtimes.
3. Train RandomForestRegressor on observed configurations.
4. Predict mean and uncertainty for unmeasured configurations.
5. Compute score = predicted_mean - 0.5 * predicted_std.
6. with 10% probability, pick the most uncertain candidate.
7. Otherwise, pick the candidate with the lowest score.
8. Measure that configuration and update the observed set.
9. Repeat until the budget is exhausted.
10. Return the best observed configuration.

The baseline Random Search uses the same valid candidate space but samples unseen configurations uniformly at random without replacement.

4. Setup

The final study evaluates three systems from the Lab 3 datasets: 7z (68,640 configurations, 8 options), LLVM (65,536 configurations, 16 options), and x264 (4,608 configurations, 10 options). All objectives are minimisation tasks.

Budgets are 10, 20, 30, 40, and 50 measurements. Each system-budget pair is repeated 30 times, and paired seeds are used so that Random Search and RF-SMBO are compared under the same random seed for each repeat.

The primary metric is the normalised gap to the optimum, where lower is better. The secondary metric is the best runtime found at a given budget. Statistical significance is assessed using a one-sided paired Wilcoxon signed-rank test.

The global optimum and worst performance values are used only for post-hoc evaluation and normalisation. They are never used by Random Search or RF-SMBO during the search process.

- Paired comparison: one run seed per system-repeat pair, shared by both algorithms.
- Search policy restriction: only valid dataset rows may be selected and measured.

5. Experiments

Figure 1 shows convergence under increasing budgets. The gap between RF-SMBO and Random Search widens as more measurements are spent, which indicates that the surrogate becomes more useful once a small observation history has been collected.

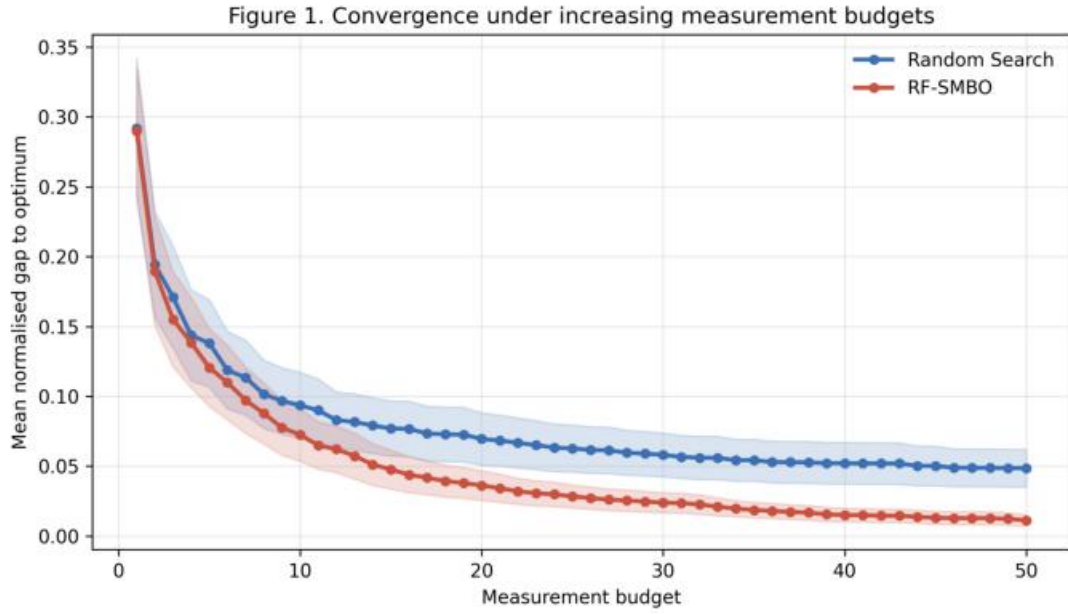


Figure 1. Mean normalised gap under increasing measurement budgets, with 95% confidence intervals.

At the final budget of 50, the overall mean normalised gap decreases from 0.0487 for Random Search to 0.0113 for RF-SMBO. This overall trend is positive, but it should be interpreted together with the per-system results because one dataset remains a clear negative case.

Table 1 reports the final-budget results at 50 measurements, including the paired-run win rate. Figure 2 then visualises the final per-system differences.

System	Random best	RF-SMBO best	Random gap	RF-SMBO gap	Gap red. (%)	p-value	win rate
7z	5405.76	6898.31	0.0029	0.0064	-123.42	0.9241	46.7%
LLVM	59231.21	53682.79	0.1351	0.0272	79.88	9.31e-10	100.0%
x264	22.9545	21.5853	0.0080	0.0002	97.90	9.10e-07	96.7%

Table 1. Final-budget results at 50 measurements. Gap reduction is relative to Random Search and win rate is the percentage of paired runs where RF-SMBO attains a lower final normalised gap.

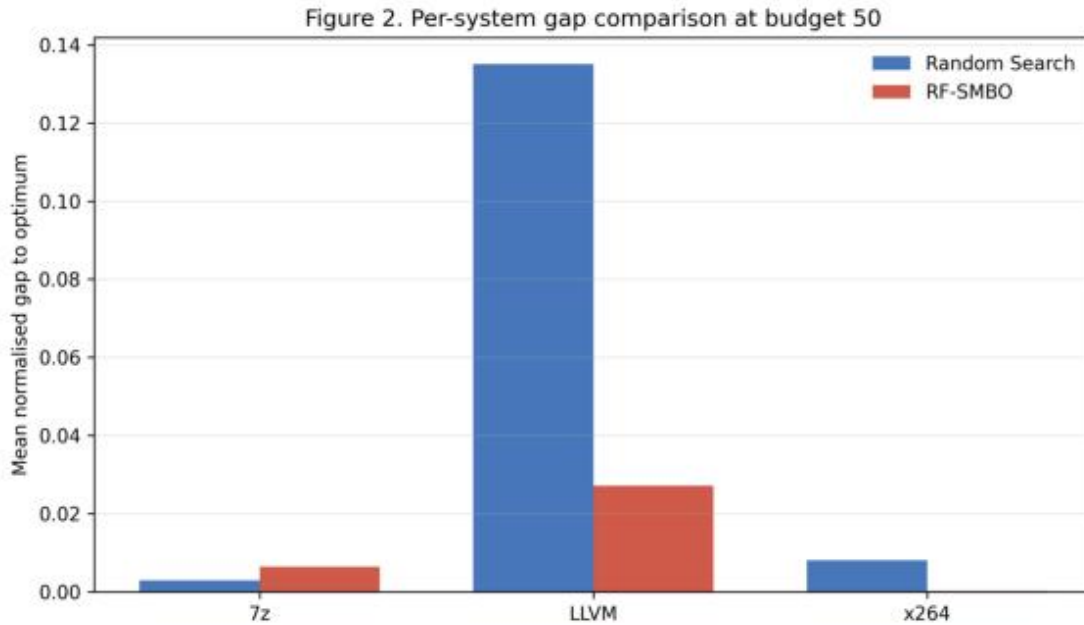


Figure 2. Per-system final-budget comparison. RF-SMBO wins clearly on LLVM and x264, but loses on 7z.

RF-SMBO improves strongly on LLVM (gap reduction 79.88%, $p = 9.31e-10$, win rate 100.0%) and x264 (gap reduction 97.90%, $p = 9.10e-07$, win rate 96.7%). On 7z, however, the mean gap increases from 0.0029 to 0.0064, so the model-guided strategy is worse than the baseline there.

6. Reflection and Conclusion

The 7z result should be treated as a valid negative result, not hidden. RF-SMBO fails on that dataset, with a higher final mean gap (0.0064) than Random Search (0.0029). This shows that the surrogate model is useful overall but not universally reliable.

- The 7z search landscape may be rugged or deceptive, making it difficult for a small surrogate to rank promising configurations.
- A very small measurement budget may not provide enough training data for stable model fitting.
- Random-forest uncertainty estimated from tree disagreement is only an approximation and may be imperfect.
- Some configuration spaces may reward broader exploration more than model-guided exploitation.

These observations strengthen the submission rather than weaken it, because they show where the method works and where it does not.

7. Future Work

- Expected Improvement or rank-based acquisition.
- An adaptive exploration coefficient instead of the fixed value 0.5.
- Comparison against local search, genetic algorithms, or A-T-EGLS.
- Multi-objective runtime-energy tuning.
- Additional systems from Lab 3.

Overall, RF-SMBO improves the mean normalised gap under limited measurement budgets, mainly due to strong gains on LLVM and x264, but it underperforms Random Search on 7z. The final submission therefore presents a promising tool together with a scientifically meaningful negative result.

Artifact

Artifact link: <https://github.com/zhaibuqun-cell/rf-smbo-configuration-tuning>.

The repository includes source code, raw CSV outputs, generated figures, requirements.pdf, manual.pdf, and replication.pdf.

References

- [1] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," Journal of Machine Learning Research, vol. 13, no. 10, pp. 281-305, 2012.
- [2] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential Model-Based Optimization for General Algorithm Configuration," LION 5 / UBC TR-2010-10, 2011.
- [3] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," NeurIPS, 2012.
- [4] P. Jamshidi and G. Casale, "An Uncertainty-Aware Approach to Optimal Configuration of Stream Processing Systems," MASCOTS, 2016.
- [5] V. Nair, T. Menzies, N. Siegmund, and S. Apel, "Using Bad Learners to find Good Configurations," ESEC/FSE, 2017.
- [6] ideas-labo, "ISE Lab 3 repository," <https://github.com/ideas-labo/ISE/tree/main/lab3>